

变量声明

`0===''` True

数组 - 引用类型

- 作用域

`let` 每次产生

`setTimeout(fun,0)` 异步操作,对象是全局对象

使用箭头函数继承外层this

- 解构赋值

不存在可能报错

- 数组对象

- `[{},{}]`

```
const users =
{name: '小明', age:20}, name: '小红', age: 22}
// map - 转换
const names = users.map(u =>u.name); //['小明', '小红']// filter - 过滤
const adults = users.filter(u => u.age >= 20);// find - 查找第一个
const found = users.find(u => u.name === '小红');// some/every -判断
users. some(u => u.age < 18);
//有未成年吗?
users.every(u =>u.age >= 18); // 都成年吗?
```

- 展开运算

- 是浅拷贝

```
const original = { user: { name: 'Tom' } };
const copy = { .. .original };
copy.user.name = 'Jerry';
console.log (original.user.name); //'Jerry' (原对象也被改了! )
//原因: 只复制了一层, 嵌套对象是引用拷贝
```

- 新特性: ? 处理null

```
const user = { address: { city: '杭州'}};//可选链? .(防止深层属性报错)
user.address?.city;user.profile?.nickname;
//「杭州!
//undefined(不会报错)
//空值合并?? (只有null/undefined 才用默认值)
const name = input R?'匿名';const name2 = input ||'匿名'; //
''和0不会被替换'和0会被替换
```

- 链式

```
实操：链式操作(3分钟) 目：计算通过考试学生的平均年龄
const students =[
  {name:name:name:
    'a',age:20,passed: true },'b', age: 22, passed:false },'c', age:21, passed: true }
//只计算 passed=true 的学生答案
2
const avgAge = students. filter(s => s. passed)map(s => s. age)
//先过滤
3
//取年龄
4
. reduce((sum, age,
arr) => sum + age / arr.length, 0);
```

- 异步

```
//2.Promise(链式调用)getUser(id)
. then(user => getOrders(user. id))
. then(orders => console. log(orders))
. catch(err => console.error(err));

// 3. async/await (推荐, 像同步代码)
async function fetchorders(userId) {
  try{
    const user = await getUser(userId);const orders = await getOrders(user.id);return
    orders;
  } catch (err) {
    console.error('出错了: ', err);
  }
}
```

```
Promise并发
//Promise.all- 全部成功才成功
const [users, orders] = await Promise.all ([fetchUsers(),
fetchOrders()
// Promise.allSettled - 等全部完成, 不管成败const results = await Promise.allsettled(
[
  fetchUsers( ),fetchOrders()
//即使失败也不影响其他的
1
```

模块化 (12 分钟)

```
// 命名导出 (多个)
export const PI = 3.14;
export function add(a, b) { return a + b; }

// 命名导入
import { add, PI } from './utils.js';
import * as utils from './utils.js'; // 全部导入

// 默认导出 (一个文件一个)
export default function main() { /* ... */ }

// 默认导入
import main from './main.js';
```

： - 小组件/工具函数：命名导出

： - 页面/模块主入口：默认导出

✅ 实操：设计导出 (5 分钟)

题目：如何组织以下函数的导出？

- formatDate() – 高频使用
- parseJSON() – 常用工具
- debounce() – 辅助函数
- 配置常量 CONSTANTS

· 参考答案：

```
1 // utils/index.js
2 export { formatDate } from './formatDate';
3 export { parseJSON } from './parseJSON';
4 export { debounce } from './debounce';
5 export { CONSTANTS } from './constants';
6 export { formatDate as default } from './formatDate'; // 默认导出最常用的
7
8 // 使用
9 import { formatDate, parseJSON, debounce } from './utils';
```

```
4
5 // 命名导入
6 import { add, PI } from './utils.js';
7 import * as utils from './utils.js'; // 全部导入
8
9 // 默认导出 (一个文件一个)
10 export default function main() { /* ... */ }
11
12 // 默认导入 (不用花括号)
13 import main from './main.js';
```

陷阱

- 使用 ===
- 错误处理

```
// 第一个参数接收值，第二个参数接收原函数返回结果并继续执行
getUser(id, function(user){
  getOrders(user.id, function(orders){
    console.log(orders.id)
  });
});
// promise
getUser(id)
  .then((user) => {getOrders(user.id)})
  .then((orders)=> console.log(orders.id))
  .catch(err =>console.error(err) )
// async/await
async function fetchOrders(id){
  try{
    const user = await getUser(id)
    const order = await getOrders(user.id)
    return order
  }catch (err){
    console.error(err)
  }
}
```

```
function getData(id) {
  return fetchUser(id)
    .then(user => fetchOrders(user.id))
    .then(orders => ({ user, orders }));
}
// 改 原箭头后的就对应一个变量接收
async function getData(id) {
  try{
    const user = await fetchUser(id)
    const order = await fetchOrders(user.id)
    return {user, order}
  }catch (err){
    console.error(err)
  }
}
```

确写法:

```
1` async function loadUserList() {
2`   try {
3       const res = await fetch('/api/users');
4       const { list = [] } = await res.json();
5       return [...list].sort((a, b) => a.age - b.age);
6`   } catch (err) {
7       console.error(err);
8       return [];
9   }
10 }
```

语法糖

```
//模板字符串
const name = 'Tom';
const msg= Hello, ${name}! `; // 支持换行和表达式//对象简写
const name = 'Tom', age = 20;
const user = { name, age }; // { name: 'Tom', age: 20 }//数组去重
const unique = [...new Set([1, 2, 2, 3])]; // [1, 2, 3]//逻辑赋值
obj.name ??='匿名'; //为 null/undefined 时才赋值//可选链+空值合并
const city = user?.address?.city ??'未知';
```

?? 会赋值

? 仅展示结果

```
var lastName = data.lastName;var displayName = firstName +
+ lastName;
var isAdult = age >= 18 ? true : false;
案
const { firstName, lastName, age } = data;
const displayName = `${firstName} ${lastName}`;
const isAdult = age >= 18;
//布尔表达式本身就是布尔值
```

TS

- 定义对象: interface, 可重复声明
- 使用联合类型: "Admin" | "user" 当枚举检查用
- Readonly() 改只读类型
- 使用 switch(){case: "circle"} 类型守卫:缩小类型范围

- case里直接写值
- Ts 是 `js` 超集

react

脚手架安装命令:

```
npm create vite@latest my-react-app -- --trmplate react-ts
```